

Comprendre l'authentification de HelpMeDraft

Guide de révision : React, Flask, MySQL, sécurité, tests et préparation au jury

Ce document reprend l'intégralité du travail réalisé sur l'inscription, la connexion, la persistance de session et la déconnexion. Il explique les choix techniques, leur lien avec le cahier des charges et les compétences du titre Concepteur Développeur d'Applications.

PÉRIMÈTRE État du projet au commit 724249e : authentification complète, interface responsive et 8 tests automatisés.

Projet de Kevin Briquet | Juin 2026

Comment utiliser ce guide

Lis d'abord les sections 1 à 4 pour comprendre le fonctionnement global. Les sections 5 à 9 expliquent les choix techniques. Termine par les questions de jury et essaie d'y répondre à voix haute sans regarder.

À SAVOIR EN 30 SECONDES React affiche et pilote les formulaires. Flask valide les données et gère la session. MySQL stocke l'utilisateur et le hash du mot de passe. Le navigateur conserve un cookie de session protégé et le renvoie à Flask à chaque requête.

Parcours de lecture

- 1. Contexte et exigences du cahier des charges
- 2. Architecture et circulation des données
- 3. Mise en place de l'environnement et de Git
- 4. Base de données et modèle utilisateur
- 5. Backend Flask : routes, validation et accès SQL
- 6. Sécurité de l'authentification
- 7. Frontend React : composants, état et appels API
- 8. Scénarios complets : inscription, connexion, session et déconnexion
- 9. Tests automatisés
- 10. Correspondance avec les compétences CDA
- 11. Limites actuelles et prochaines améliorations
- 12. Questions-réponses de jury
- 13. Glossaire et checklist de révision

Résultat obtenu

Réalisé	Inscription	Validation, e-mail unique, consentement, hash et session immédiate.
Réalisé	Connexion	Comparaison sécurisée du mot de passe et création de session.
Réalisé	Persistance	La session est reconnue après actualisation via /api/me.
Réalisé	Déconnexion	La session est supprimée côté Flask.
Réalisé	Qualité	Interface responsive, accessibilité de base et 8 tests automatisés.

1. Contexte et exigences du cahier des charges

Le cahier des charges de LexiCorp demande une application web sécurisée et accessible permettant de rédiger, stocker et améliorer des documents avec une IA. La gestion utilisateur est le socle : sans identité fiable, il est impossible d'associer correctement les dossiers, documents, interactions et quotas.

Exigences directement couvertes

Réalisé	Inscription / connexion / déconnexion	Routes <code>/api/register</code> , <code>/api/login</code> et <code>/api/logout</code> .
Réalisé	Espace personnel sécurisé	Session signée, route <code>/api/me</code> et décorateur <code>@login_required</code> .
Réalisé	Hash des mots de passe	<code>generate_password_hash</code> et <code>check_password_hash</code> de Werkzeug.
Réalisé	Protection contre les injections SQL	Requêtes paramétrées avec des marqueurs <code>%s</code> .
Réalisé	Rôles utilisateur / administrateur	Champ <code>role</code> présent ; l'inscription crée un utilisateur <code>role = 0</code> .
Réalisé	Quota initial	<code>quota_restant</code> initialisé à 20 lors de l'inscription.
Partiel	Accessibilité RGAA	Labels, focus clavier, aria-live, contraste et responsive ; audit formel à venir.
Partiel	RGPD	Consentement explicite à l'inscription ; politique complète et traçabilité à formaliser.
À venir	Récupération de mot de passe	Non implémentée dans ce bloc.
À venir	Audit sécurité complet	CSRF dédié, limitation des tentatives et HTTPS de production à finaliser.

CHOIX D'ARCHITECTURE Le cahier des charges recommande React, Flask et MySQL : le projet suit exactement cette pile. Il mentionne aussi JWT ou sessions ; nous avons choisi les sessions Flask, explicitement admises dans la partie sécurité (« hash + JWT ou sessions »).

Pourquoi commencer par l'authentification ?

- Chaque document doit appartenir à un utilisateur identifié.
- Les quotas IA doivent être décomptés sur le bon compte.
- Le back-office devra distinguer les utilisateurs standards des administrateurs.
- L'historique et les données personnelles ne doivent pas être accessibles sans authentification.

2. Architecture et circulation des données

Pendant le développement, trois services coopèrent. React et Flask utilisent deux serveurs distincts, tandis que MySQL tourne via WampServer.

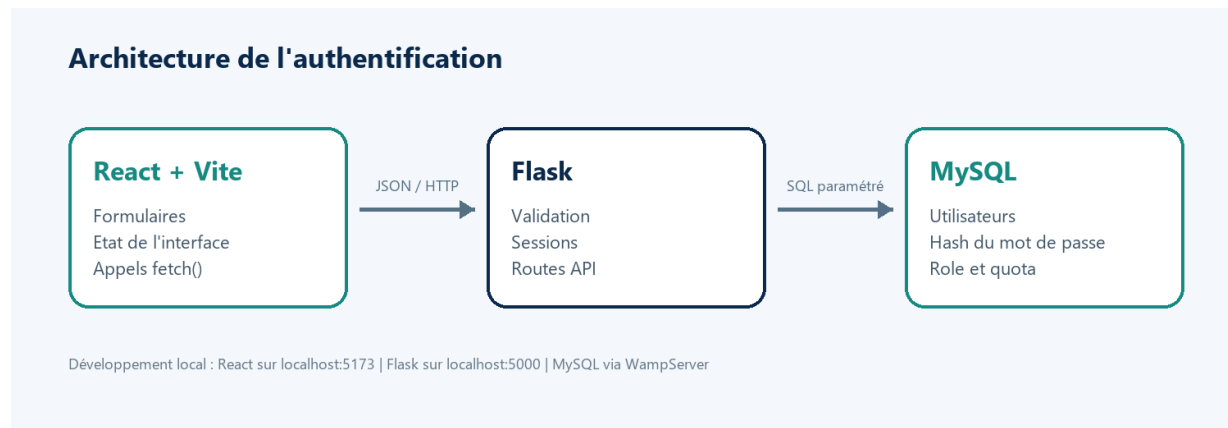


Figure 1 — Architecture actuelle du bloc d'authentification

Rôle de chaque partie

Élément	Responsabilité
React / Vite	Afficher l'interface, mémoriser les champs, déclencher les événements et appeler l'API.
Flask	Recevoir le JSON, appliquer les règles métier, interroger MySQL et gérer la session.
MySQL	Conserver les utilisateurs, le hash du mot de passe, le rôle, la date et le quota.
Navigateur	Afficher React et conserver le cookie de session envoyé par Flask.
Git / GitHub	Versionner le code, sécuriser les étapes et fournir un historique démontrable.

Un flux typique

1. L'utilisateur remplit le formulaire React.
2. React transforme les données en JSON avec `JSON.stringify()`.
3. `fetch()` envoie une requête HTTP POST à Flask.
4. Flask valide les données, puis exécute une requête SQL paramétrée.
5. Flask renvoie une réponse JSON avec un code HTTP adapté.
6. React met à jour l'interface sans recharger toute la page.

VOCABULAIRE Une API est ici le contrat entre le frontend et le backend. Les routes `/api/...` définissent les actions disponibles et le format attendu des échanges.

3. Environnement de travail et versionnement

Structure créée

```
HelpMeDraft/
├── .gitignore
├── backend/
│   ├── .env
│   ├── app.py
│   ├── requirements.txt
│   ├── test_auth.py
│   └── venv/
└── frontend/
    ├── package.json
    └── src/
```

Pourquoi un environnement virtuel Python ?

Le dossier venv isole les bibliothèques de HelpMeDraft. Flask, Flask-Cors, python-dotenv et mysql-connector-python peuvent avoir des versions spécifiques sans perturber les autres projets installés sur la machine.

Commandes principales du backend

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install Flask mysql-connector-python python-dotenv Flask-Cors
python -m pip freeze > requirements.txt
```

Variables d'environnement

Les identifiants MySQL et la clé de session ne sont pas écrits dans le code. Ils sont chargés depuis backend/.env avec python-dotenv. Ce fichier est exclu de Git.

Noms de variables attendus — les vraies valeurs secrètes ne doivent jamais être documentées

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=helpmedraft
DB_USER=root
DB_PASSWORD=...
SECRET_KEY=...
APP_ENV=development
```

Git et GitHub

- git init a créé le dépôt local.
- La branche principale a été renommée main.
- Le dépôt local a été relié au dépôt GitHub HelpMeDraft.
- Les fonctionnalités ont été sauvegardées par petits commits cohérents.
- .gitignore exclut venv, .env, node_modules et __pycache__.

BONNE PRATIQUE Un commit doit représenter une étape compréhensible et testée. L'historique actuel montre l'initialisation de Flask, la connexion MySQL, le formulaire, les sessions, les tests et l'inscription.

4. Base de données et modèle utilisateur

La table utilisateur existait dans le MCD. Elle est centrale pour le bloc d'authentification et sera référencée par les dossiers, documents et interactions.

Colonne	Type / contrainte	Utilité
Id_utilisateur	INT, PK, AUTO_INCREMENT	Identifiant technique unique.
nom / prenom	VARCHAR(50), NOT NULL	Identité affichée dans l'espace personnel.
email	VARCHAR(50), UNIQUE, NOT NULL	Identifiant de connexion ; unicité garantie en base.
mot_de_passe_hash	VARCHAR(255), NOT NULL	Hash sécurisé, jamais le mot de passe original.
role	TINYINT(1), NOT NULL	0 = utilisateur standard ; 1 = administrateur.
date_inscription	DATETIME	Date de création du compte.
quota_restant	INT	Nombre de requêtes IA encore disponibles.

Correction importante apportée

Le champ mot_de_passe_hash était initialement en VARCHAR(100) et acceptait NULL. Il a été passé en VARCHAR(255) NOT NULL pour accueillir les formats de hash actuels et empêcher la création d'un compte sans mot de passe.

```
ALTER TABLE utilisateur
MODIFY mot_de_passe_hash VARCHAR(255) NOT NULL;
```

Pourquoi l'unicité de l'e-mail est vérifiée deux fois ?

Flask vérifie d'abord si l'e-mail existe pour renvoyer un message clair. La contrainte UNIQUE de MySQL reste la protection définitive, notamment contre deux inscriptions simultanées. Une version de production devra aussi intercepter l'erreur d'intégrité SQL.

POINT JURY Une validation frontend améliore l'expérience utilisateur, une validation backend protège l'application, et une contrainte de base garantit l'intégrité des données.

5. Backend Flask : routes et règles métier

Route	Responsabilité	Codes
POST /api/register	Créer un utilisateur, hacher le mot de passe et ouvrir une session.	201 / 400 / 409
POST /api/login	Vérifier l'e-mail et le mot de passe, puis ouvrir une session.	200 / 400 / 401
GET /api/me	Retourner l'utilisateur associé à la session active.	200 / 401
POST /api/logout	Supprimer la session.	200
GET /api/test	Vérifier que Flask répond.	200
GET /api/test-db	Vérifier la connexion à la base.	200

Inscription

- Normalisation de l'e-mail avec `strip().lower()`.
- Contrôle des champs obligatoires et des longueurs.
- Validation simple du format de l'e-mail.
- Confirmation du mot de passe.
- Règle minimale : 8 caractères, majuscule, minuscule et chiffre.
- Consentement obligatoire.
- Vérification de l'unicité de l'e-mail.
- Hash du mot de passe, insertion SQL, commit et création de session.

Connexion

Exemple de requête paramétrée

```
cursor.execute(
    """
    SELECT Id_utilisateur, nom, prenom, email, mot_de_passe_hash, role
    FROM utilisateur
    WHERE email = %s
    """,
    (email,),
)
```

Le marqueur `%s` est remplacé par le connecteur MySQL. La valeur n'est pas concaténée dans la chaîne SQL : une saisie malveillante reste une donnée et ne devient pas une commande.

Décorateur de protection

Une route privée réutilise le contrôle de session

```
@app.get("/api/me")
@login_required
def get_current_user():
    ...
```

Le décorateur `@login_required` centralise la règle « une session doit contenir `user_id` ». Il évite de recopier le même contrôle dans chaque future route de documents ou de profil.

6. Sécurité de l'authentification

Hash et chiffrement ne sont pas la même chose

Un chiffrement est réversible avec une clé. Un hash de mot de passe est conçu pour ne pas être déchiffré. Lors de la connexion, `check_password_hash` recalcule une valeur à partir du mot de passe saisi et vérifie qu'elle correspond au hash stocké.

RÈGLE ABSOLUE Le mot de passe original ne doit jamais être enregistré, renvoyé par l'API, écrit dans les logs ou commité dans Git.

Session Flask

Protection	Rôle
<code>SECRET_KEY</code>	Signe le cookie et empêche sa falsification. Elle reste dans <code>.env</code> .
<code>HttpOnly</code>	Interdit au JavaScript du navigateur de lire le cookie.
<code>SameSite=Lax</code>	Réduit l'envoi du cookie lors de requêtes provenant d'un autre site.
Secure en production	Le cookie ne circule qu'en HTTPS quand <code>APP_ENV=production</code> .
Durée de 8 heures	La session permanente expire après une durée définie.
<code>session.clear()</code>	Supprime l'ancienne session à la connexion et à la déconnexion.
<code>user_id</code> uniquement	La session ne conserve qu'un identifiant minimal, pas le mot de passe.

Point subtil à maîtriser

Par défaut, Flask stocke la session dans un cookie signé côté navigateur. Signé ne veut pas dire chiffré : l'utilisateur ne peut pas modifier le contenu sans invalider la signature, mais il ne faut pas y placer de donnée secrète. Conserver seulement `user_id` est donc adapté.

Ce qui reste à renforcer

- Ajouter un mécanisme CSRF explicite pour les actions sensibles.
- Limiter les tentatives de connexion répétées et journaliser les événements suspects.
- Forcer HTTPS et une configuration de production sans debug.
- Intercepter proprement les erreurs MySQL et garantir la fermeture via `try/finally`.
- Ajouter la récupération de mot de passe avec jeton temporaire.
- Prévoir éventuellement une session côté serveur pour des besoins plus avancés.

RÉPONSE HONNÊTE AU JURY Le bloc est sécurisé pour le développement et démontre les principes essentiels. Il ne faut pas prétendre qu'il est prêt pour Internet tant que CSRF, limitation des tentatives, HTTPS, gestion d'erreurs et configuration de production ne sont pas finalisés.

7. Frontend React : ce qu'il faut comprendre

Le CSS n'est qu'une partie du frontend. Pour le titre CDA, il faut surtout être capable d'expliquer comment React gère les données, les événements et la communication avec l'API.

Le composant App

App est actuellement le composant principal. Il contient les deux formulaires, l'écran connecté et la logique d'appel au backend. Pour une application plus grande, il sera découpé en composants LoginForm, RegisterForm, AuthTabs et UserSession.

Concept React	Utilisation dans HelpMeDraft
useState	Mémoire une valeur qui évolue : e-mail, mot de passe, mode, utilisateur, chargement.
useEffect	Exécute checkSession une fois au chargement pour interroger /api/me.
Événement onChange	Récupère la nouvelle valeur d'un champ et met à jour l'état.
Événement onSubmit	Intercepte l'envoi du formulaire et empêche le rechargement de page.
Affichage conditionnel	Affiche connexion, inscription ou écran connecté selon authMode et user.
fetch	Envoie une requête HTTP et attend la réponse JSON du backend.
credentials: include	Autorise l'envoi et la réception du cookie de session.

Champ contrôlé

```
<input
  value={email}
  onChange={(event) => setEmail(event.target.value)}
  required
/>
```

La valeur affichée vient de l'état React. À chaque frappe, onChange met cet état à jour. React devient la source de vérité du formulaire : on parle de champ contrôlé.

Accessibilité déjà intégrée

- Chaque champ possède un label lié par htmlFor et id.
- Les attributs autocomplete aident le navigateur et les technologies d'assistance.
- Le message utilise role=status et aria-live pour être annoncé.
- Les boutons ont des libellés clairs et un focus visible.
- L'interface s'adapte aux écrans étroits et respecte prefers-reduced-motion.
- La validation finale reste côté Flask : HTML required ne suffit pas à sécuriser l'application.

8. Scénarios complets

Scénario A — Création d'un compte

1. React affiche les champs prénom, nom, e-mail, mot de passe, confirmation et consentement.
2. `handleSubmit` détecte `authMode = register` et appelle `POST /api/register`.
3. Flask valide toutes les données et refuse les incohérences avec un code 400.
4. Flask cherche l'e-mail ; s'il existe, la réponse est 409 Conflict.
5. Le mot de passe est haché et l'utilisateur est inséré avec `role = 0` et `quota = 20`.
6. Flask place le nouvel identifiant dans `session['user_id']` et renvoie 201 Created.
7. React stocke l'objet `user` et affiche immédiatement l'écran connecté.

Scénario B — Connexion

8. React envoie l'e-mail et le mot de passe à `POST /api/login`.
9. Flask recherche l'utilisateur par e-mail avec une requête paramétrée.
10. `check_password_hash` compare la saisie au hash de MySQL.
11. En cas d'échec, Flask renvoie le même message générique « Identifiants incorrects ».
12. En cas de succès, Flask renouvelle la session et renvoie les informations publiques.

Scénario C — Actualisation de la page

`useEffect` appelle `GET /api/me` avec `credentials: include`. Le navigateur joint le cookie. `@login_required` valide la présence de `user_id`, Flask recharge l'utilisateur depuis MySQL et React réaffiche l'écran connecté.

Scénario D — Déconnexion

React appelle `POST /api/logout`. Flask exécute `session.clear()`. React supprime ensuite son état `user` et revient au formulaire de connexion.

Codes HTTP à retenir

Code	Sens dans le projet
200 OK	Connexion, lecture de session ou déconnexion réussie.
201 Created	Compte utilisateur créé.
400 Bad Request	Données absentes ou invalides.
401 Unauthorized	Authentification absente ou identifiants incorrects.
409 Conflict	L'e-mail existe déjà.

9. Tests automatisés

Le fichier backend/test_auth.py utilise unittest, inclus dans Python. Le client de test Flask simule un navigateur et les mocks remplacent temporairement MySQL.

Test	Résultat attendu
Connexion sans champs	400 et message obligatoire.
Mauvais mot de passe	401 et refus de connexion.
Bonne connexion	200 et user_id présent dans la session.
Mot de passe d'inscription faible	400.
E-mail déjà utilisé	409.
Inscription réussie	201, commit SQL, hash vérifiable et session créée.
Accès à /api/me sans session	401.
Déconnexion	user_id supprimé de la session.

Pourquoi simuler MySQL ?

Un test unitaire doit être rapide, reproductible et isolé. MagicMock fournit une fausse connexion et une fausse réponse SQL. Les tests ne créent ni ne suppriment aucun compte réel.

```
@patch("app.get_db_connection")
def test_login_creates_user_session(self, mocked_connection):
    mocked_connection.return_value = self.create_fake_connection(user)
    response = self.client.post("/api/login", json={...})
    self.assertEqual(response.status_code, 200)
```

Commande de test

```
python -m unittest -v test_auth.py
```

DISTINCTION Les 8 tests actuels sont principalement unitaires. Il faudra ensuite ajouter des tests d'intégration avec une base dédiée et des tests de parcours frontend.

10. Correspondance avec les compétences CDA

Le bloc d'authentification ne couvre pas à lui seul tout le titre, mais il fournit déjà des preuves concrètes pour plusieurs compétences. Les statuts ci-dessous évitent de surévaluer l'avancement.

Réalisé	CP1 — Installer et configurer l'environnement	Node, npm, Python, venv, dépendances, WampServer et variables d'environnement.
Réalisé	CP2 — Développer des interfaces utilisateur	Formulaires React contrôlés, responsive, messages d'état et accessibilité de base.
Réalisé	CP3 — Développer des composants métier	Règles d'inscription, validation, session, rôle et quota initial.
Partiel	CP4 — Contribuer à la gestion d'un projet	Git/GitHub et commits réguliers ; planification et documentation projet à poursuivre.
Partiel	CP5 — Analyser les besoins et maquetter	Traduction du CDC en parcours utilisateur ; maquettes formelles à archiver.
Partiel	CP6 — Définir l'architecture logicielle	Séparation React / Flask / MySQL ; découpage en couches encore à réaliser.
Partiel	CP7 — Concevoir une base relationnelle	MCD fourni, contraintes PK/UNIQUE/FK et adaptation du champ de hash.
Réalisé	CP8 — Développer l'accès aux données SQL	SELECT/INSERT paramétrés, commit et fermeture des ressources.
Réalisé	CP9 — Préparer et exécuter des tests	8 tests automatisés isolés avec unittest et mock.
Partiel	CP10 — Préparer et documenter le déploiement	requirements.txt et .gitignore présents ; Docker et .env.example à venir.
Partiel	CP11 — Contribuer à une démarche DevOps	Dépôt distant et historique ; pipeline CI/CD à venir.

Ce que tu peux montrer au jury

- Le schéma React → Flask → MySQL et un parcours de connexion.
- La table utilisateur et les contraintes d'intégrité.
- Une route Flask avec requête SQL paramétrée.
- La configuration du cookie de session.
- Un composant React avec useState, useEffect et fetch.
- L'exécution des 8 tests automatisés.
- L'historique Git avec des commits fonctionnels.

11. Limites actuelles et prochaines améliorations

Limites techniques assumées

Constat	Amélioration prévue
App.jsx est volumineux	Créer des composants LoginForm, RegisterForm, AuthLayout et UserCard.
app.py concentre toutes les routes	Introduire des Blueprints, services métier et repositories d'accès aux données.
URL du backend écrite dans React	Utiliser une variable VITE_API_URL.
Connexion MySQL répétitive	Créer un gestionnaire de contexte ou une couche repository avec try/finally.
Pas de gestion centralisée des erreurs	Ajouter des error handlers Flask et des réponses JSON cohérentes.
Pas de récupération de mot de passe	Créer des jetons temporaires à durée limitée et un envoi d'e-mail.
CSRF non finalisé	Ajouter un jeton CSRF ou une stratégie adaptée aux routes mutables.
Pas de limitation des tentatives	Mettre en place rate limiting et journalisation.
Tests frontend absents	Ajouter Vitest / React Testing Library et un test de parcours.

Prochaine architecture cible

```

backend/
├── app/
│   ├── routes/auth.py
│   ├── services/auth_service.py
│   ├── repositories/user_repository.py
│   ├── validators/auth_validator.py
│   └── extensions.py
├── tests/
└── run.py

frontend/src/
├── components/auth/
├── pages/
├── services/api.js
└── App.jsx

```

LIEN AVEC LE CDA Ce futur découpage rendra particulièrement visible la compétence CP6 : une application organisée en couches, avec responsabilités séparées et dépendances maîtrisées.

12. Questions-réponses de jury

1. Pourquoi avoir choisi React ?

React facilite la création d'interfaces composées et réactives. L'état permet de mémoriser les champs et l'utilisateur connecté, puis l'affichage se met à jour sans recharger la page.

2. Qu'est-ce qu'un composant React ?

C'est une fonction qui retourne une partie d'interface. Elle peut recevoir des propriétés, posséder un état et être réutilisée. App est actuellement le composant principal.

3. À quoi sert useState ?

useState conserve une valeur entre les rendus du composant. Quand le setter est appelé, React recalcule l'interface avec la nouvelle valeur.

4. À quoi sert useEffect ici ?

Il déclenche une vérification de session une seule fois au chargement de l'application en appelant /api/me.

5. Qu'est-ce qu'un champ contrôlé ?

Sa valeur vient de l'état React et chaque saisie met cet état à jour avec onChange. React reste la source de vérité.

6. Pourquoi frontend et backend ont-ils deux serveurs ?

En développement, Vite sert React sur le port 5173 et Flask expose l'API sur le port 5000. Cette séparation accélère le développement ; le déploiement pourra les réunir derrière une même origine.

7. À quoi sert CORS ?

Le navigateur bloque par défaut certaines requêtes entre origines différentes. Flask-Cors autorise précisément le frontend localhost:5173 à appeler les routes /api.

8. Pourquoi credentials: include ?

Cette option autorise fetch à recevoir puis renvoyer le cookie de session lors des appels entre React et Flask.

9. Pourquoi avoir choisi une session plutôt qu'un JWT ?

La session Flask convient bien à une application web classique. Elle simplifie la révocation et permet un cookie HttpOnly. Un JWT serait pertinent pour plusieurs clients ou une architecture distribuée, mais augmente la complexité de stockage et de révocation.

10. Que contient la session ?

Uniquement `user_id`. Flask signe le cookie avec `SECRET_KEY`. Aucune donnée sensible, aucun mot de passe ni hash n'y sont stockés.

11. Un cookie Flask est-il chiffré ?

Par défaut il est signé, pas chiffré. La signature empêche la modification, mais le contenu ne doit pas être considéré secret.

12. Pourquoi `SECRET_KEY` doit-elle rester secrète ?

Une personne qui la connaît pourrait fabriquer des cookies de session valides et usurper un utilisateur.

13. Pourquoi hacher le mot de passe ?

Pour ne jamais conserver le secret original. En cas de fuite de base, un hash lent et salé rend la récupération des mots de passe beaucoup plus coûteuse.

14. Quelle différence entre hash et chiffrement ?

Le chiffrement est réversible avec une clé. Le hash de mot de passe est à sens unique ; on vérifie une saisie sans retrouver le mot de passe original.

15. Comment évitez-vous l'injection SQL ?

Les requêtes utilisent des paramètres `%s` fournis séparément. Le connecteur traite la saisie comme une donnée et non comme du SQL exécutable.

16. Pourquoi valider côté React et côté Flask ?

React améliore l'expérience utilisateur, mais peut être contourné. Flask est la frontière de confiance et doit toujours revalider les données.

17. Pourquoi une contrainte `UNIQUE` sur l'e-mail ?

Elle garantit l'intégrité au niveau de la base, même si deux requêtes concurrentes essaient de créer le même compte.

18. Que signifient 400, 401, 409 et 201 ?

400 : requête invalide ; 401 : authentification absente ou refusée ; 409 : conflit avec une ressource existante ; 201 : ressource créée.

19. À quoi sert `@login_required` ?

Ce décorateur centralise la protection des routes privées. Il vérifie `user_id` dans la session avant d'exécuter la route.

20. Pourquoi renvoyer un message générique pour une mauvaise connexion ?

Dire si l'e-mail existe aiderait un attaquant à recenser les comptes. « Identifiants incorrects » évite cette fuite d'information.

21. Comment sont testées les routes ?

Le client de test Flask simule les requêtes HTTP. `unittest.mock` remplace la connexion MySQL afin de tester la logique sans modifier la vraie base.

22. Pourquoi ne pas tester directement la vraie base ?

Les tests unitaires doivent être isolés et reproductibles. Une base dédiée sera utilisée plus tard pour des tests d'intégration.

23. Quelles protections manquent encore ?

CSRF explicite, limitation des tentatives, HTTPS obligatoire, gestion d'erreurs centralisée, récupération de mot de passe et configuration de production.

24. Comment respectez-vous le RGPD ?

Le formulaire demande un consentement explicite. Les mots de passe ne sont jamais conservés en clair. Il faudra encore documenter les finalités, durées de conservation, droits et traitements IA.

25. Comment amélioreriez-vous l'architecture ?

Je séparerais routes, services métier, validation et repositories SQL, et je découperais `App.jsx` en composants spécialisés.

26. Qu'apporte Git au projet ?

Git fournit un historique, permet de revenir à un état stable, facilite la collaboration et apporte des preuves d'avancement pour la soutenance.

13. Glossaire et checklist de révision

Terme	Définition
API	Ensemble de routes permettant au frontend de communiquer avec le backend.
Backend	Partie serveur qui applique les règles métier et accède aux données.
CORS	Règles du navigateur autorisant ou refusant les appels entre origines.
Cookie	Petite donnée conservée par le navigateur et envoyée au serveur.
Décorateur	Fonction Python qui ajoute un comportement à une autre fonction.
Frontend	Interface exécutée dans le navigateur.
Hash	Empreinte à sens unique utilisée pour protéger un mot de passe.
HTTP	Protocole utilisé pour les requêtes et réponses entre React et Flask.
JSON	Format texte structuré utilisé dans les échanges de l'API.
Mock	Objet simulé utilisé pour isoler un test.
Requête paramétrée	Requête SQL où les valeurs sont séparées du texte SQL.
Session	Mécanisme permettant au serveur de reconnaître un utilisateur entre plusieurs requêtes.
State / état	Données mémorisées par un composant React.
venv	Environnement Python isolant les dépendances du projet.

Checklist avant de présenter ce bloc

- ☐ Je peux dessiner React → Flask → MySQL sans regarder.
- ☐ Je sais expliquer useState, useEffect, fetch et l'affichage conditionnel.
- ☐ Je sais expliquer pourquoi le mot de passe est haché et jamais chiffré en base.
- ☐ Je sais expliquer le cookie de session, SECRET_KEY, HttpOnly et SameSite.
- ☐ Je sais montrer une requête SQL paramétrée et expliquer l'injection SQL.
- ☐ Je connais le rôle de /api/register, /api/login, /api/me et /api/logout.
- ☐ Je connais les codes 200, 201, 400, 401 et 409.
- ☐ Je peux lancer les 8 tests et expliquer pourquoi MySQL est simulé.
- ☐ Je peux citer honnêtement les limites actuelles et les améliorations prévues.
- ☐ Je sais relier ce travail aux CP1, CP2, CP3, CP8 et CP9 du titre CDA.

FORMULATION POUR CONCLURE À L'ORAL « J'ai construit un parcours d'authentification complet reliant une interface React, une API Flask et une base MySQL. J'ai sécurisé les mots de passe, les requêtes SQL et la session, puis j'ai automatisé huit scénarios de test. Je connais aussi les limites restantes avant une mise en production. »

Sources du guide

- Cahier des charges « CDC - HelpMeDraft - MNS », LexiCorp, avril 2025.
- MCD / structure MySQL fournie pour HelpMeDraft.
- Code du dépôt HelpMeDraft au commit 724249e.
- Référentiel de compétences CDA fourni dans le support de formation.